

OLED

Experiment Objective

In this experiment, you will learn how to drive the 0.91-inch OLED display (SSD1306, 128x32 pixels) on the MicroROS-V2 control board. Through this example, you will master the configuration of the ESP32's I2C master mode, the basic principles of the SSD1306 display driver, and how to display text information on the OLED.

Experiment Principle

1. Keywords

Keyword	Description
I2C bus	A serial communication protocol that uses two lines, SCL (clock) and SDA (data), for communication between devices
SSD1306	An OLED display driver chip that supports I2C/SPI interfaces, with resolutions of 128x64/128x32
Display buffer	An area in memory used to store pixel data; the SSD1306 128x32 screen needs 512 bytes
Pixel coordinate	A point position on the screen in pixels; x ranges from 0 to 127, y from 0 to 31
Font library (font)	Predefined character dot-matrix data used to display text on the screen
I2C device address	The unique address that identifies each device on the I2C bus; the OLED address is 0x3C

2. Technical Architecture

The program communicates with the SSD1306 OLED through the ESP32's I2C master mode. During initialization, the I2C bus is configured (SCL=GPIO2, SDA=GPIO1, 400 KHz), and then a complete initialization command sequence is sent to the OLED. The display uses a frame-buffer approach: the content to be displayed is first written into the memory buffer, and then sent to the OLED at once via `OLED_Refresh()`. Key files are located in `~/esp/base_samples/oled/`, including `main/main.c` (main program entry), `components/i2c_master/i2c_master.h` (I2C pin parameters) and `components/i2c_master/i2c_master.c` (I2C low-level driver), `components/oled/bsp_oled.h` (OLED hardware parameters), `components/oled/oled.h` (display API declarations) and `components/oled/oled.c` (display API implementation).

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB Type-C data cable	✓ Required

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Confirm the OLED display on the control board is functioning properly

3. Compile and Flash

```
cd ~/esp/base_samples/oled
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After compiling and flashing successfully, the serial monitor will output:

```
hello yahboom
I (xxx) OLED_MAIN: Nice to meet you!
```

At the same time, the OLED screen displays:

- First line: `OLED init waiting..`
- Second line: a value starting from -128, incremented by 1 every second, displayed in a format such as `value_count: -128, value_count: -127 ...`

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/oled/main/main.c`
- I2C driver: `~/esp/base_samples/oled/components/i2c_master/i2c_master.h, i2c_master.c`
- OLED driver: `~/esp/base_samples/oled/components/oled/bsp_oled.h, oled.h, oled.c`

1. Main Program Logic

`main/main.c` is the program entry, and the flow is as follows:

- 1. Print the startup message `"hello yahboom"` and a welcome log
- 2. Call `OLED_12832_Init()` to initialize the I2C and OLED display
- 3. Call `OLED_Draw_Line()` to display `"OLED init waiting.."` on the first line, and clear the screen
- 4. Enter the main loop, updating the count value displayed on the second line every second, cycling from -128 up to 127

2. Key Components

I2C Master Driver

`components/i2c_master/i2c_master.h` — I2C pin and parameter definitions:

Parameter definition file: `~/esp/base_samples/oled/components/i2c_master/i2c_master.h`

```
#define I2C_MASTER_GPIO_SCL      2      // I2C clock pin GPIO2
#define I2C_MASTER_GPIO_SDA     1      // I2C data pin GPIO1
#define I2C_MASTER_NUM          0      // I2C port number I2C_NUM_0
#define I2C_MASTER_FREQ_HZ      400000 // I2C clock frequency 400 KHz
#define I2C_MASTER_TIMEOUT_MS   1000   // I2C timeout 1000 ms
```

`components/i2c_master/i2c_master.c` — I2C master driver implementation, core functions:

Function	Description
<code>I2C_Master_Init()</code>	Initialize I2C master mode, configure SCL (GPIO2) and SDA (GPIO1), frequency 400 KHz
<code>I2C_Master_Delete()</code>	Uninstall the I2C driver
<code>I2C_Master_Read_Byte()</code> / <code>I2C_Master_Read()</code>	Read one byte / a series of data from the I2C device
<code>I2C_Master_Write_Byte()</code> / <code>I2C_Master_Write()</code>	Write one byte / a series of data to the I2C device

I2C initialization code:

Source file: `~/esp/base_samples/oled/components/i2c_master/i2c_master.c`

```
i2c_config_t conf = {
    .mode = I2C_MODE_MASTER,
    .sda_io_num = 1,          // GPIO1
    .scl_io_num = 2,          // GPIO2
    .sda_pullup_en = GPIO_PULLUP_ENABLE,
    .scl_pullup_en = GPIO_PULLUP_ENABLE,
    .master.clk_speed = 400000,
};
i2c_param_config(I2C_NUM_0, &conf);
i2c_driver_install(I2C_NUM_0, conf.mode, 0, 0, 0);
```

OLED Display Driver

`components/oled/bsp_oled.h` — display parameter definitions:

Parameter definition file: `~/esp/base_samples/oled/components/oled/bsp_oled.h`

```
#define SSD1306_WIDTH 128      // OLED width 128 pixels
#define SSD1306_HEIGHT 32     // OLED height 32 pixels
```

The I2C address of the OLED is **0x3C**. When writing commands, the register address is 0x00; when writing data, the register address is 0x40:

```
#define OLED_ADDRESS 0x3C
#define SSD1306_WRITECOMMAND(command) I2C_Master_Write_Byte(OLED_ADDRESS, 0x00, (command))
#define SSD1306_WRITEDATA(data)       I2C_Master_Write_Byte(OLED_ADDRESS, 0x40, (data))
```

Display buffer: the size is `128 * 32 / 8 = 512` bytes

`components/oled/oled.h` — upper-layer API function declarations:

Function	Description
<code>OLED_12832_Init()</code>	Initialize the I2C and OLED display (complete initialization flow)
<code>OLED_Clear()</code>	Clear the screen content (fill with black)
<code>OLED_Refresh()</code>	Refresh the screen, sending the buffer content to the OLED
<code>OLED_Draw_String(char *data, uint8_t x, uint8_t y, bool clear, bool refresh)</code>	Display a string at the specified position; x and y are character coordinates (uses the Font_7x10 font, 7x10 pixels per character)
<code>OLED_Draw_Line(char *data, uint8_t line, bool clear, bool refresh)</code>	Display a string on the specified line, supports lines 1~3

OLED initialization sequence:

`OLED_Init()` sends a complete initialization command sequence: turn off the display -> set the clock divide ratio -> set the multiplex ratio (0x1F corresponds to 32 lines) -> set the display offset -> set the start line -> enable the charge pump -> set the COM pins configuration -> segment remapping -> reverse the scan direction -> set the contrast -> set the pre-charge period -> set the VCOMH level -> clear the screen -> turn on the display.

Display line mapping:

`OLED_Draw_Line()` maps the line number (1~3) to the character Y coordinate (0~2). Each character is 10 pixels tall, so three lines can display a total of 30 pixels (leaving some spare space on the 32-pixel-high screen):

- Line 1: Y-axis pixel position 0
- Line 2: Y-axis pixel position 10
- Line 3: Y-axis pixel position 20

`components/oled/oled_fonts.h` — font definitions:

Four fonts are supported: 6x8, 7x10 (default), 11x18, and 16x26. `OLED_Draw_String()` uses the `Font_7x10` font by default, and each line can display up to about 18 English characters ($128/7 \approx 18$).

Common Problems

Problem	Cause	Solution
OLED displays nothing	Wrong I2C pins or mismatched OLED address	Confirm SCL=GPIO2, SDA=GPIO1, and the I2C address is 0x3C
Blurred or garbled display content	I2C communication rate too high	Check whether <code>I2C_MASTER_FREQ_HZ</code> is 400000; try lowering the frequency
OLED stays on with no changes	Display buffer not refreshed	Check whether the <code>refresh</code> parameter of <code>OLED_Draw_Line()</code> is set to true
Characters displayed incompletely	Incorrect font selection or coordinates out of bounds	<code>OLED_Draw_Line()</code> only supports lines 1~3; the x coordinate range is 0~17